



Energy and robustness trade-offs in adaptive neural mmWave channel estimation on edge devices

Eric Meneses-Albalá¹ · Saúl Villaescusa¹ · José M. Badía² · Germán León² · Carmen Botella-Mascarell¹ · Sandra Roger¹

Received: 14 October 2025 / Accepted: 14 January 2026
© The Author(s) 2026

Abstract

The evolution toward 6G will continue to leverage massive multiple-input multiple-output and millimeter-wave systems, which demand accurate angle-of-arrival (AoA) and angle-of-departure (AoD) estimation. While several deep learning models have demonstrated strong performance for this task, their accuracy, like that of most estimation methods, is often degraded by hardware non-idealities, which can be further exacerbated by time-varying operational factors such as component aging and adverse weather, among others. Building on a pre-trained U-Net architecture with demonstrated competitive performance for AoA/AoD estimation, we first propose an adaptation mechanism based on fine-tuning with impairment-augmented data. Specifically, we simulate hardware imperfections by introducing random phase errors in the antenna elements, ranging from mild fluctuations to severe signal distortions. The U-Net model with adaptation capabilities is then implemented on an NVIDIA Jetson Orin Nano device, a compact edge platform with heterogeneous computing resources. To this end, we design a co-execution strategy that performs AoA/AoD estimation (inference) on the CPU while simultaneously fine-tuning the model on the GPU, thus enabling continuous model adaptation to changing environmental or hardware conditions while preserving real-time inference performance. Experimental results show that impairment-aware fine-tuning effectively counters hardware degradation, particularly under significant phase impairments. In such scenarios, the fine-tuned model consistently preserves or even improves estimation accuracy, reducing the Root Mean Square Error (RMSE) by approximately 3.6% and increasing the Probability of Detection (P_D) by up to 1 percentage point compared to the base model. Furthermore, a detailed energy-performance analysis demonstrates that while maximum frequency settings reduce training time by over 11×, they also increase power consumption by more than 5×, with optimal energy efficiency achieved at mid-range CPU and high GPU frequencies. This work establishes the feasibility of concurrent training and inference on resource-constrained heterogeneous hardware, paving the way for resilient and autonomous edge intelligence in future 6G systems.

Extended author information available on the last page of the article

Keywords 5G and 6G communications · Edge computing · MIMO · Deep learning

1 Introduction

As a key enabler of 5G high-performance features, the millimeter-wave (mmWave) spectrum is expected to be a fundamental component of future 6G networks [1, 2]. However, communications in these bands face significant challenges, including large propagation losses and atmospheric absorption. To address these, massive multiple-input multiple-output (MIMO) systems employ large antenna arrays to perform beamforming, creating narrow high-gain links [3]. This directional approach requires precise alignment between the transmitter and receiver, making beam management procedures, specifically the estimation of angle-of-departure (AoD) and angle-of-arrival (AoA), essential for maintaining connectivity [4]

While traditional signal processing techniques typically exploit the sparsity of the mmWave channel [3, 5], Deep Learning (DL) has emerged as a powerful alternative [6]. A primary advantage of DL is the ability to shift computational complexity to the offline training phase, enabling low-latency inference on resource-constrained edge devices. In our recent work [7], we established the theoretical performance of ResNet and U-Net architectures. While [7] included a sensitivity analysis regarding hardware impairments, it was limited to evaluating static models trained on ideal data, concluding that retraining is necessary to fully mitigate performance loss. Furthermore, while our preliminary deployment study [8] validated the inference energy of these models on an NVIDIA Jetson Orin Nano, it did not explore the computational feasibility of performing this required fine-tuning process on-device. Consequently, relying solely on the static baselines from these prior works risks significant performance degradation when facing real-world impairments.

In practice, performance is often compromised by a multitude of factors, including hardware imperfections like phase shifter errors [9, 10], as well as component aging and environmental shifts [11, 12]. To address this, we propose a fine-tuning strategy that adapts the pre-trained U-Net model to hardware-impaired channels. Distinct from the inference-only focus of [8], this work advances the methodology by characterizing the energy trade-offs of a concurrent execution model: the CPU handles real-time inference of angles, while the GPU simultaneously performs the fine-tuning process. This approach allows for continuous, hardware-specific adaptation, a critical capability for meeting the energy-efficiency demands of resource-constrained Edge AI devices [13].

This execution strategy aligns with the principles of managing heterogeneous resources to minimize energy consumption, a concept explored theoretically in [14]. We distinguish our contribution from [14] by moving beyond theoretical minimization. Instead, we provide a critical experimental validation and profiling on an embedded edge device (Jetson Orin Nano). We demonstrate that while a fully automated closed-loop training system is outside the current scope, the device possesses the necessary resources to run inference and training in parallel, effectively mitigating the impact of phase impairments.

The key contributions of this work are:

- We fine-tune a CNN-based AoA/AoD estimator (U-Net) with hardware-impaired channel data, yielding an improvement in probability of detection (P_D) and root-mean-square error (RMSE) of the estimated angles compared to baseline models trained only on ideal data.
- We propose and validate a heterogeneous CPU/GPU execution mode on an embedded edge device, enabling inference and fine-tuning to run concurrently.
- We provide a comprehensive characterization of the trade-offs in energy consumption and execution time associated with this co-execution.

The remainder of the paper is structured as follows. Section 2 describes the system model and hardware impairments. Section 3 details the proposed DL approach and fine-tuning strategy. Section 4 presents the fine-tuned models performance evaluation results. Section 5 discusses the embedded implementation and experimental profiling on the Jetson Orin Nano. Finally, Sect. 6 concludes with a summary of findings and future directions.

2 System model

This section details the mmWave MIMO channel model and the signal acquisition process. We also introduce the modeling of hardware impairments, a core focus of this study.

2.1 Channel model and signal acquisition

We consider a single-user mmWave MIMO system where both the transmitter (Tx) and receiver (Rx) are equipped with uniform linear arrays, comprising n_t and n_r antenna elements, respectively. The channel is characterized by L distinct scatterers, each contributing a single propagation path. For each path $l \in \{1, \dots, L\}$, the complex channel coefficient is denoted by α_l , along with its AoA ψ_l and AoD ϕ_l . The complete parametric channel model is expressed as [15]:

$$\mathbf{H}(\boldsymbol{\theta}) = \sqrt{n_t n_r} \sum_{l=1}^L \alpha_l \mathbf{a}_r(\psi_l) \mathbf{a}_t^H(\phi_l), \quad (1)$$

where $\boldsymbol{\theta} \triangleq [|\alpha_1|, \angle\alpha_1, \phi_1, \psi_1, \dots, |\alpha_L|, \angle\alpha_L, \phi_L, \psi_L]^T$ is the parameter vector. Here, $|\alpha_l|$ and $\angle\alpha_l$ denote the magnitude and phase of each channel coefficient, respectively. The path coefficients α_l are assumed to be independent and identically distributed (i.i.d.) random variables following $\mathcal{CN}(0, \sigma_a^2/L)$, while the AoAs (ψ_l) and AoDs (ϕ_l) are uniformly distributed within $[0, 2\pi]$.

The array response vectors for the Tx and Rx antennas, assuming half-wavelength antenna spacing, are given by:

$$\mathbf{a}_l(\phi_l) = \frac{1}{\sqrt{n_l}} [1, e^{-j\pi \cos \phi_l}, \dots, e^{-j\pi(n_l-1) \cos \phi_l}]^T, \quad (2)$$

$$\mathbf{a}_r(\psi_l) = \frac{1}{\sqrt{n_r}} [1, e^{-j\pi \cos \psi_l}, \dots, e^{-j\pi(n_r-1) \cos \psi_l}]^T. \quad (3)$$

Estimating the channel $\mathbf{H}(\boldsymbol{\theta})$ is equivalent to determining the parameters in $\boldsymbol{\theta}$. The sparsity of mmWave channels [16] simplifies this task.

2.1.1 Pilot-based training phase

The estimation procedure begins with an open-loop pilot-based training phase. The beam search space is defined by a codebook with P and Q predefined codewords/directions for the Tx and Rx, respectively. A pilot symbol s (known to both Tx and Rx) is transmitted across a subset of $P \leq N_{\max}$ and $Q \leq N_{\max}$ spatial directions. In the analog beamforming (ABF) scenario considered, both Tx and Rx utilize a single radio-frequency chain, with beamforming and combining operations performed in the analog domain [5]. Beamforming/combining vectors are computed to match the channel response [17]. For each $\{q, p\}$ direction pair, the received signal $y_{q,p}$ is given by:

$$y_{q,p} = \sqrt{\rho} \mathbf{w}_q^H \mathbf{H} \mathbf{f}_p s + \mathbf{w}_q^H \mathbf{n}, \quad (4)$$

where $\rho \in \mathbb{R}^+$ is the transmit power, and $\mathbf{w}_q^H \mathbf{n}$ represents the additive complex white Gaussian noise term. During training, s is set to 1 for simplicity. The collected $y_{q,p}$ measurements form the complex-valued observation matrix $\mathbf{Y}$:

$$\mathbf{Y} = \begin{bmatrix} y_{0,0} & y_{0,1} & \cdots & y_{0,P-1} \\ y_{1,0} & y_{1,1} & \cdots & y_{1,P-1} \\ \vdots & \vdots & \ddots & \vdots \\ y_{Q-1,0} & y_{Q-1,1} & \cdots & y_{Q-1,P-1} \end{bmatrix} = \sqrt{\rho} \mathbf{G}(\boldsymbol{\theta}) + \mathbf{N}. \quad (5)$$

Here, $\mathbf{N}$ is the noise matrix and $\mathbf{G}(\boldsymbol{\theta})$ encodes the channel parameters. This observation matrix $\mathbf{Y}$, in our case, is treated as a 2D image for subsequent deep learning-based AoA/AoD estimation, which is framed as a supervised image-to-image translation problem [8].

2.2 Hardware impairments modeling

In practical ABF systems, ideal phase shifts are difficult to achieve due to hardware imperfections in phase shifters, which can introduce small deviations from the intended phase values and thus distort the desired beam pattern [9]. To evaluate the impact of such imperfections on angular estimation performance, we model the phase shift error associated with the i -th antenna element, denoted by δ_i , as a random variable. This error is uniformly distributed over the range $[-\delta_{\max}, \delta_{\max}]$,

where $\delta_{\max} \in (0^\circ, 180^\circ]$ quantifies the maximum phase deviation. This probabilistic representation captures the variability inherent in non-ideal hardware implementations [9].

Assuming these errors are independent across antenna elements, the effective phase shift applied to the i -th element of both the Tx and Rx antenna arrays is perturbed by δ_i . This directly alters the array responses in Eqs. (2) and (3), leading to deviations from the ideal channel characteristics. These deviations manifest as distortions in the signal's complex structure, altering the relationship between the two components. Our preliminary analysis indicates that while small values of $\delta_{\max}$ (e.g., 1° to 5°) introduce minor fluctuations, larger values (e.g., $> 15^\circ$) cause substantial distortion to the phase information encoded in the input.

In our experimental evaluation, we consider various levels of phase-shifter error to assess the robustness of the proposed methods under severe hardware impairments [10]. The parameter $\delta_{\max}$ is set to 1° , 2° , and 5° . These values are selected to represent a spectrum of realistic hardware impairments: $\delta_{\max} = 1^\circ$ corresponds to high-quality implementations, while $\delta_{\max} = 5^\circ$ represents a pessimistic scenario to test robustness under severe conditions. We exclude higher values as they correspond to infeasible hardware specifications where the signal degradation would be excessive.

3 Deep-learning-based mmWave channel estimation

Our approach for estimating AoAs and AoDs is based on a U-Net architecture, building upon our previous work [7, 8]. The methodology comprises three stages. First, the observation matrix is transformed into a structured input representation suitable for convolutional processing. Second, a U-Net encoder–decoder network is employed to reconstruct the original ground truth observation matrix, yielding a denoised representation that facilitates peak detection for estimating the dominant AoAs and AoDs. Third, the model is trained and subsequently fine-tuned to compensate for noise induced by hardware impairments.

3.1 Data representation

The complex-valued observation matrix $\mathbf{Y}$ (as described in Sect. 2.1) is first separated into its real ($\mathbf{Y}_{\Re}$) and imaginary ($\mathbf{Y}_{\Im}$) components. To standardize input dimensions across different antenna configurations and preserve spatial structure, nearest-neighbor interpolation [18] is applied to both components, upsampling them to a uniform size (e.g., 64×64 for our base model). These interpolated real and imaginary components are then stacked to form the network input, typically of size $Q' \times P' \times 2$.

In contrast, the U-Net does not output discrete beam indices or a downsampled angular codebook. Instead, it produces a continuous representation $\mathbf{Z} \in \mathbb{R}^{M \times N}$, where $M = N = 256$ in our implementation. Each pixel represents a spatial frequency sample over an extended range beyond $[0, 2\pi]$ to handle boundary

effects [7]. This high-resolution frequency map allows sub-beam localization, enabling an AoA/AoD prediction beyond the angular granularity imposed by the original 16×16 codebook.

Propagating paths manifest as localized peaks in $\mathbf{Z}$, which are extracted to estimate AoA/AoD directions. To extract these directions, we use a blob-detection procedure inspired by OpenCV's `SimpleBlobDetector`. This method identifies blobs, estimates their centroids with subpixel precision, and ranks them by their underlying intensity, yielding ordered AoA/AoD candidates that map consistently onto the continuous frequency domain representation. The complete procedure follows the blob detection framework described in [7]. The U-Net model, while larger than other CNNs like ResNet, is still efficiently processed on embedded systems [8].

3.2 Network architecture

The process begins with pre-processing of the observation matrix, followed by its input into the neural network, and finally, post-processing of the network's output to identify angular parameters.

This process is conceptualized as a four-stage pipeline, illustrated on the left side of Fig. 1. It begins with the separated observation matrix, which is subsequently interpolated and used as the input to the U-Net. The network reconstructs an estimated ground-truth representation from the interpolated matrix, producing the output $\mathbf{Z} \in \mathbb{R}^{M \times N}$. This output is then provided to the blob-detection module that identifies peak locations and maps them to their corresponding AoAs and AoDs.

The right side of Fig. 1 illustrates the internal architecture of the U-Net, as detailed in [7], and highlights the following three principal components.

1. *The encoder*: A contracting path consisting of five depth blocks (labeled D-Block 1 to 5). These blocks progressively downsample the input spatial resolution (from $Q' \times P'$ down to $Q'/32 \times P'/32$) while increasing the feature depth to 512 channels to capture high-level context.
2. *The bottleneck*: A dense block connects the deepest encoder layer to the decoder. This block flattens the feature map to integrate global information before reshaping it for reconstruction.
3. *The decoder*: An expansive path comprising five upsampling blocks (labeled U-Block 5 to 1). These blocks sequentially restore the spatial dimensions of the feature maps.

Additionally, skip connections (represented by dashed arrows) concatenate the output of each D-Block with the input of the corresponding U-Block, preserving fine-grained spatial details for precise angular localization.

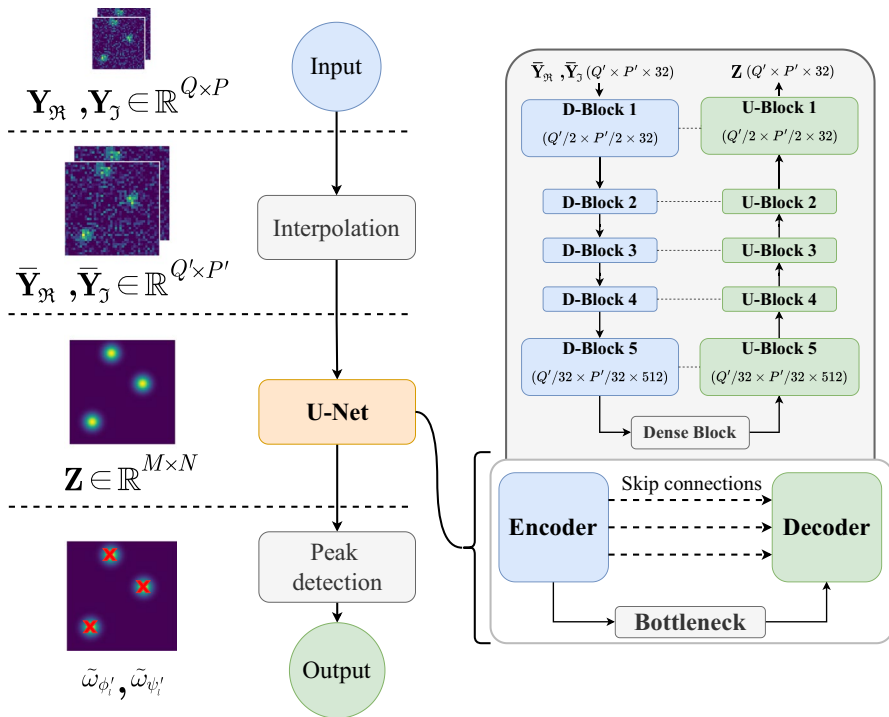


Fig. 1 Main stages of the proposed AoAs and AoDs estimation

3.3 Training configuration

During the training phase, the network is optimized to map the input observation matrix $\mathbf{Y}$ to a high-resolution ground-truth matrix $\mathbf{X}$. This target matrix $\mathbf{X}$ is composed of a sum of 2D Gaussian functions, where each Gaussian represents an ideal channel path with specific AoA and AoD centers and a predefined standard deviation (e.g., $\sigma_1 = \sigma_2 = 0.2$) [19]. The model is trained by minimizing the Mean Squared Error (MSE) between the network's output $\mathbf{Z}$ and the ground truth $\mathbf{X}$, as defined by the loss function:

$$\text{Loss} = \frac{1}{MN} \sum_{n=0}^{N-1} \sum_{m=0}^{M-1} (\mathbf{Z}_{m,n} - \mathbf{X}_{m,n})^2. \quad (6)$$

3.4 Fine-tuning strategy

While the base U-Net model is initially trained with ideal channel representations, its performance can degrade when exposed to real-world hardware

impairments. To address this, we employed a systematic fine-tuning strategy, adapting the pre-trained model to handle additional non-ideal characteristics, as defined in Sect. 2.2.

Specifically, we generated a synthetic dataset with phase shift errors ($\delta_{\max} \in 1^\circ, 2^\circ, 5^\circ$) to fine-tune the model and simulate hardware impairments. The validation set included multiple conditions defined by the number of paths, SNR, and maximum phase error. Hyperparameters and training protocols are summarized in Table 1.

During fine-tuning, a fraction of encoder layers was frozen to preserve generic spatial features learned from the impairment-free model. Encoder blocks primarily capture low- and mid-level spatial statistics, which remain informative under moderate distortions, and freezing them limits parameter drift and reduces overfitting.

Decoder layers remained trainable, as they govern reconstruction and must adapt to altered phase statistics and impairment-induced deviations. We evaluated freezing ratios of 0%, 20%, 50%, 80%, 90% to regulate the balance between adaptation and retention. This approach prevents excessive modification of stable encoder representations while allowing the reconstruction pathway to specialize to the impaired input distribution.

Figure 2 shows the evolution of training metrics during fine-tuning with 90% of the encoder layers frozen, with similar trends observed for the other freezing ratios. Panels (a) and (b) display the training and validation loss, respectively, computed using the objective function in Eq. (6), while panels (c) and (d) show the corresponding training and validation Mean Absolute Error (MAE). The validation loss curves show a consistent downward trend across impairment levels, though higher loss is observed on $\delta_{\max} = 5$ due to the increased noise induced by the more severe impairments.

Table 1 Hyperparameters and training configurations for fine-tuning the pre-trained U-Net on synthetic data with hardware impairments

Category	Setting
Base model	Pre-trained U-Net (no simulated impairments)
Training data	10,000 synthetic samples with hardware impairments
Validation data	4,800 samples across a grid of conditions (L , SNR, PQ , $\delta_{\max}$)
Phase error levels	$\delta_{\max} \in \{1^\circ, 2^\circ, 5^\circ\}$
Batch size	32
Optimizer	Adam
Initial learning rate	1×10^{-4}
Learning rate schedule	Reduce by 25% after 20 epochs w/o validation improvement
Minimum learning rate	1×10^{-6}
Early stopping	Triggered if validation loss stagnates for 80 epochs
Max epochs	800
Layer freezing	Encoder frozen at $\{0\%, 20\%, 50\%, 80\%, 90\%\}$
Model selection	Best checkpoint by minimum validation loss
Loss function	Defined in Eq. 6

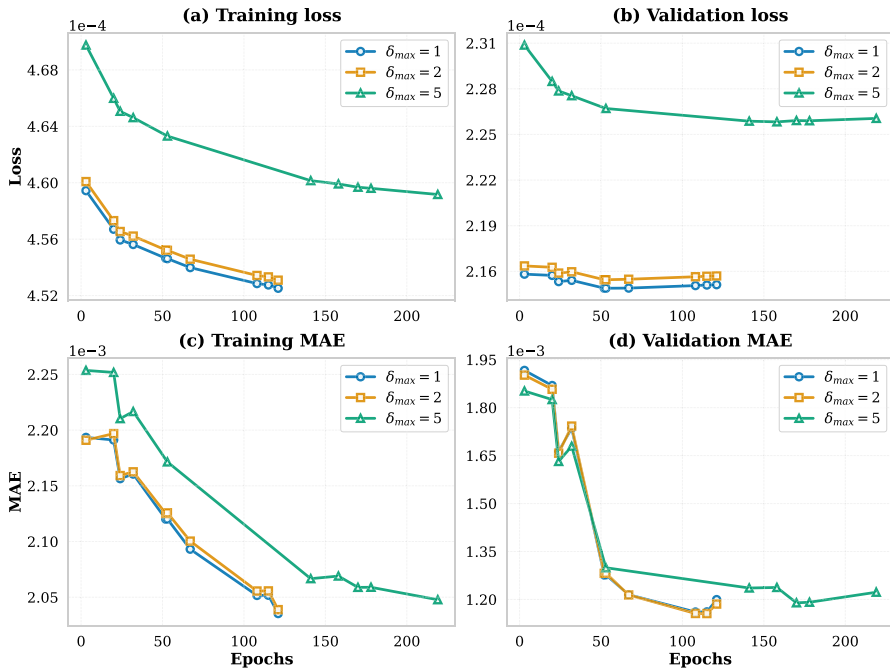


Fig. 2 Evolution of training metrics during transfer learning with 90% frozen layers across three configurations ($\delta_{\max} = 1, 2, 5$). **a** Training loss and **b** validation loss. **c** Training MAE and **d** validation MAE

Notably, the validation metrics are consistently lower than the training metrics. This behavior is attributed to the difference in data generation strategies: while the training set consists of 10,000 samples generated via continuous random sampling, the validation set comprises 4,800 samples organized in a fixed parameter grid. The specific grid points selected for validation present a slightly more favorable average distribution for the model than the broader variance encountered during training.

Despite this offset, the trajectories follow a similar downward trend. While $\delta_{\max} = 5$ shows slight overfitting after epoch 150, we only kept the best-performing model validation loss-wise at epoch 158. In contrast, the lower impairment scenarios ($\delta_{\max} = 1, 2$) converged significantly earlier, achieving their minimum validation loss at epoch 53 on both.

4 Performance evaluation of fine-tuned models

This section presents an evaluation of the performance of the fine-tuned U-Net models under various hardware impairment conditions. Our analysis focuses on comparing the estimation accuracy achieved by the original base model against our selected best-performing fine-tuned variants, specifically those trained with 90% of their encoder layers frozen.

It is crucial to understand that each fine-tuned model is specialized for a particular impairment range, defined by the maximum phase shift error, $\delta_{\max}$. For instance, the “Freeze 90% for $\delta_{\max} = 1^\circ$ ” model was fine-tuned exclusively with data generated under a $\delta_{\max} = 1^\circ$ phase shift error. The original base model (whose pre-trained weights are available in [20]), by contrast, was not adapted to specific impairments but was evaluated against test data incorporating various $\delta_{\max}$ levels to assess its degradation. This setup allowed for a direct comparison, quantifying the benefits of targeted fine-tuning under specific synthetic hardware conditions.

For this evaluation, the AoA/AoD estimation accuracy is assessed through two key metrics. The first is the RMSE of the angle measurements, calculated as:

$$\text{RMSE} = \sqrt{\frac{1}{|\mathcal{N}|} \sum_{i \in |\mathcal{N}|} (\vartheta_i - \hat{\vartheta}_i)^2}, \quad (7)$$

where $\mathcal{N}$ denotes the set of successfully detected angles, and ϑ_i represents any true AoA or AoD, while $\hat{\vartheta}_i$ is its corresponding estimate. A detection is considered successful if its associated RMSE is less than or equal to 1° .

The second metric is the P_D , which represents the proportion of successfully detected angles out of the total number of actual paths present in the scene, evaluated across different SNRs.

Table 2 presents a consolidated overview of the performance metrics for the base model and the specialized Freeze 90% fine-tuned models across different $\delta_{\max}$ values and SNR levels. While the absolute variations appear subtle due to the high precision of the base model, they serve as a critical validation of the proposed edge deployment strategy. The results confirm that the fine-tuned models predominantly match or outperform the base model, acting as a precision optimizer. Notably, as the impairment severity increases (e.g., $\delta_{\max} = 5^\circ$), the fine-tuned model mitigates degradation, maintaining a P_D of 0.94 at 25 dB compared

Table 2 Base model vs. Freeze 90 fine-tuned model. Equal or superior results with the fine-tuned model are shown in bold

SNR (dB)	$\delta_{\max} = 1$				$\delta_{\max} = 2$				$\delta_{\max} = 5$			
	Base		Fine-tuned		Base		Fine-tuned		Base		Fine-tuned	
	RMSE	P_D	RMSE	P_D	RMSE	P_D	RMSE	P_D	RMSE	P_D	RMSE	P_D
-10	0.54	0.23	0.54	0.22	0.55	0.22	0.53	0.22	0.55	0.22	0.54	0.22
-5	0.51	0.47	0.51	0.46	0.51	0.46	0.51	0.45	0.51	0.46	0.51	0.46
0	0.44	0.67	0.44	0.69	0.44	0.67	0.44	0.68	0.45	0.67	0.44	0.68
5	0.38	0.81	0.38	0.81	0.38	0.81	0.38	0.81	0.38	0.81	0.38	0.81
10	0.31	0.89	0.31	0.88	0.32	0.88	0.31	0.88	0.32	0.88	0.32	0.88
15	0.27	0.92	0.26	0.93	0.26	0.92	0.26	0.92	0.28	0.91	0.27	0.91
20	0.23	0.95	0.23	0.95	0.23	0.94	0.23	0.94	0.25	0.93	0.25	0.93
25	0.21	0.95	0.21	0.95	0.21	0.95	0.22	0.95	0.24	0.93	0.24	0.94

to 0.93 for the base. For lower impairment levels ($\delta_{\max} \leq 2^\circ$), slight fluctuations are observed, suggesting the base model already possesses significant robustness against mild hardware imperfections.

Quantitatively, the results demonstrate that on-device adaptation yields tangible benefits, becoming particularly relevant under the higher impairment regime of $\delta_{\max} = 5^\circ$. In this scenario, the fine-tuned model effectively preserves or selectively enhances the P_D and RMSE. Specifically, at 15 dB SNR, the RMSE improves from 0.28 to 0.27, corresponding to a relative gain of approximately 3.6%. Furthermore, at 25 dB SNR, the fine-tuning yields an absolute increase in P_D of 1 percentage point (from 0.93 to 0.94), ensuring robust detection rates despite the increased phase errors. Overall, this confirms that the adaptation process successfully stabilizes performance against significant hardware imperfections, yielding gains of up to 3.6% relative to the base model.

Finally, these results highlight the efficiency of the approach. Rather than requiring full retraining, the method demonstrates that freezing 90% of the model parameters is sufficient to preserve baseline accuracy and prevent degradation under hardware impairments. By maintaining high performance in the majority of cases and yielding measurable gains in others, the approach confirms that the network can adapt without catastrophic forgetting, offering a favorable trade-off between computational cost and reliability.

5 Edge device performance and resource utilization

The practical viability of deep learning models for real-time mmWave channel estimation heavily relies on their efficient operation on resource-constrained edge devices. As highlighted in the introduction, hardware impairments are an inevitable challenge in real-world deployments, degrading the performance of models not specifically adapted to them. While Sect. 4 has demonstrated that fine-tuning with impairment-augmented data maintains or improves the robustness of U-Net models, the practical application of this adaptation in standalone edge systems remains a critical challenge.

Therefore, in this section, we evaluate the feasibility of implementing such an adaptive learning strategy directly on an edge device like the NVIDIA Jetson Orin Nano, specifically focusing on scenarios involving the concurrent execution of inference and training tasks. The aim is to characterize the energy consumption of the edge device in a hypothetical scenario in which both the edge device and the antenna system could autonomously compensate for hardware impairments.

5.1 Experimental environment

For this research, our experimental setup employed the NVIDIA Jetson Orin Nano Developer Kit as the primary hardware platform. These compact yet powerful modules are built on an 8 nm process and feature the Orin System-on-Chip (SoC). Each Orin SoC integrates a hexa-core ARM Cortex-A78AE CPU, with a 1.5 MB L2

cache and a 4 MB L3 cache, alongside an NVIDIA Ampere GPU comprising four Streaming Multiprocessors, 512 CUDA cores, and 16 Tensor Cores. The System-on-Module (SOM) includes the SoC and 4 GB of 64-bit LPDDR5 memory, providing a 34 GB/s bandwidth dynamically shared between the CPU and GPU. The Jetson Orin Nano supports configurable power profiles, offering a low-power 7 W mode for energy efficiency and a higher 10 W mode for enhanced performance. Figure 3 shows the main components of the Jetson Orin Nano module, including its Orin SoC.

In our experiments, we investigated how different fixed frequencies for both the CPU and GPU affect the module's overall power consumption and task execution time. This configuration was achieved using the NVIDIA Board Support Package (BSP) [21]. By default, the CPU and GPU frequencies are managed by the *on-demand* governor, which dynamically adjusts their clock speeds based on workload demand. However, for our tests, we manually set fixed frequencies using the BSP.

Moreover, we integrated the enhanced *pmlib* framework [22] to precisely quantify the resulting power utilization, which continuously monitors and logs power data at 10 Hz directly from the device's integrated sensors. By using this framework, we analyzed aggregated power consumption across several critical subsystems, including the entire module and the combined processing load from the CPU, GPU, and other components. This granular monitoring provides a comprehensive understanding of power distribution within the device under varying frequency configurations.

From these measurements, we derived the energy consumption, expressed in joules (J), by multiplying the average power consumption by the duration of each specific task.

5.2 Energy characterization of co-running tasks

The experiments in this section involve two concurrent processes: one performing continuous inference on the CPU with a batch size of one image, and another

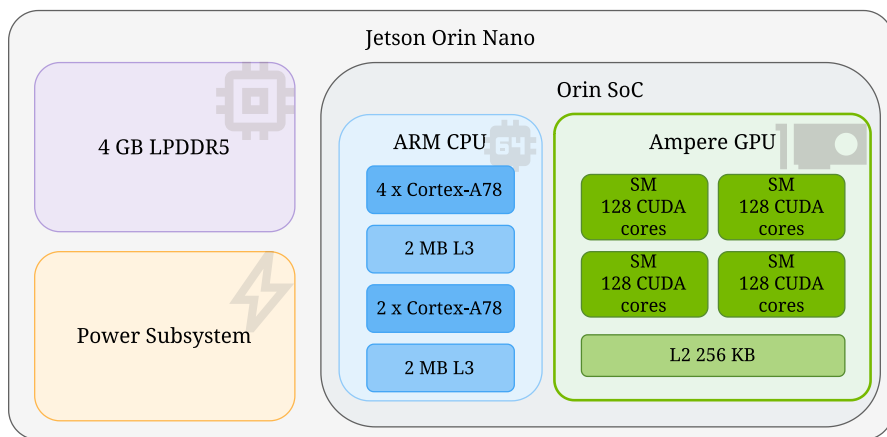


Fig. 3 Main components of the Jetson Orin Nano module

conducting model training on the GPU. We used TensorFlow's device placement features to assign each process to a specific SoC device. To ensure stable measurements, we first executed a one-epoch warm-up phase. Performance was then measured over two subsequent training epochs, recording the total execution time and average power consumption. From these measurements, we calculated the training time per epoch and per sample. The reported power results reflect the dynamic power of the tasks, calculated by subtracting the platform's idle power consumption from the total power measured during execution.

Training time is highly dependent on batch size. As shown in Fig. 4, a batch size of one image underutilizes the GPU's parallel processing capabilities and makes it difficult to hide the latency of data transfers between the CPU and GPU. Increasing the batch size improves GPU core utilization and allows computation to overlap with data transfers. Consequently, the training time per epoch is significantly reduced when using batch sizes of two or four images, with diminishing returns for larger sizes. The results also demonstrate that GPU training time is unaffected by the concurrent inference process on the CPU, indicating that the SoC's resources are sufficient for both tasks to operate without contention. The training process imposes a low load on the CPU, saturating a single core while leaving the other five idle. In contrast, the inference process utilizes all six CPU cores at approximately 50% load.

Figure 4 also shows that larger batch sizes do not lead to a disproportionate increase in power consumption, suggesting they are more energy-efficient for training. Furthermore, concurrent CPU and GPU operation increases the SOM's average power consumption by 20 – 30% compared to GPU-only training. Based on these findings, a batch size of 16 images was selected for all subsequent experiments, as it provided the optimal balance between training time and energy consumption.

The power consumption of individual SOM components fluctuates based on the active phase of the training and inference workload, such as data transfers

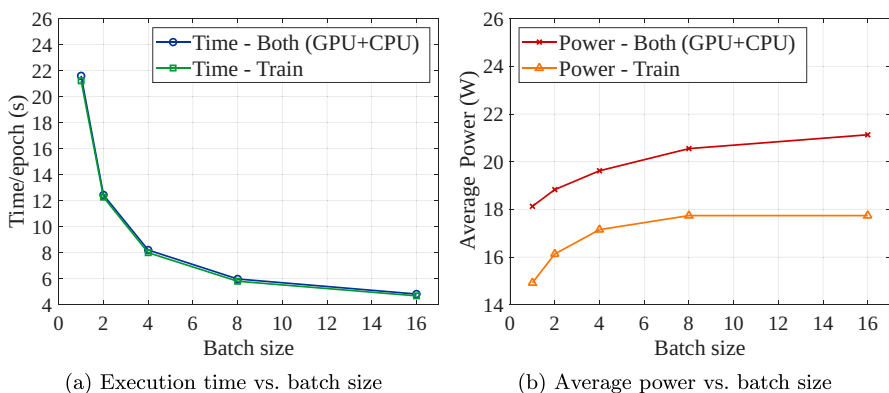


Fig. 4 Execution time per epoch in seconds (a) and average power consumption in watts (b) to train the model in the GPU with (both) and without (train) performing the inference process in the CPU

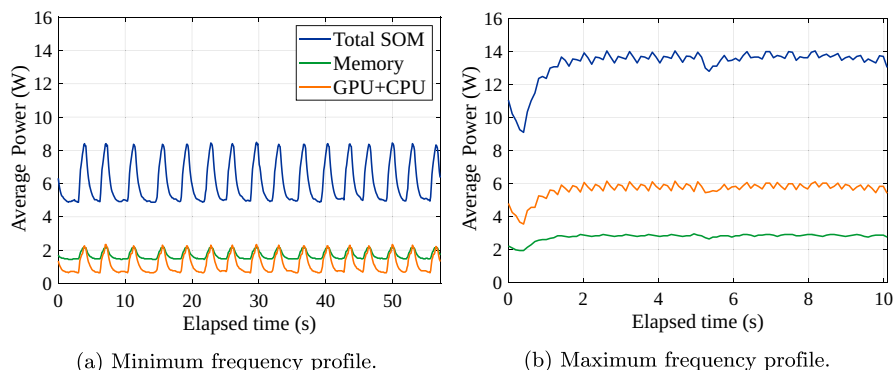


Fig. 5 Evolution of the average power consumption (in watts) of the training and inference processes over two epochs (16 batches total) at minimum (a) and maximum (b) CPU and GPU frequencies. Lines show the total power of the Jetson Orin Nano module (SOM) and its main power domains (Memory and GPU+CPU)

versus GPU computation. By sampling power at 10 Hz across different power domains, we can observe this evolution. Figure 5 illustrates these power dynamics at minimum and maximum CPU and GPU frequencies.

Operating at maximum frequencies reduces the total training time from 57.2 seconds to 5.1 seconds. This performance gain, however, corresponds to a substantial increase in power consumption, particularly for the CPU and GPU, while memory power shows a more modest rise. The figure also reveals the cyclic nature of power consumption during the training of the 16 batches. At minimum frequencies, this pattern is distinct, with clear alternations between data transfer and computation phases for each batch. The pattern is less pronounced but still visible at maximum frequencies, including a slight drop in consumption between the two epochs.

To find the optimal frequency settings, we measured performance across all available CPU and GPU frequency combinations. Figure 6 details the effect of these frequencies on training time per sample. The results show that low CPU frequencies dramatically increase training time. This time decreases sharply as the CPU frequency increases from 115.2 MHz to 422.4 MHz, with further increases up to the maximum (1510.4 MHz) yielding only marginal improvements. Conversely, GPU frequency has a minimal impact at low CPU frequencies. However, at higher CPU frequencies, training time consistently decreases as GPU frequency increases.

As shown in Fig. 7, increasing CPU and GPU frequencies leads to a corresponding rise in the platform's average power consumption. For example, power draw increases from 1.63 watts at minimum frequencies to 8.49 watts at maximum frequencies, more than 5 times higher. A distinct region of lower power consumption is evident at low CPU frequencies, largely independent of GPU frequency. This low-power region corresponds directly to the high-processing-time region observed in Fig. 6.

To determine the optimal frequency combination for overall efficiency, we calculated the energy consumed per sample as the product of processing time and average power. The processing time is defined as the average time required to process one

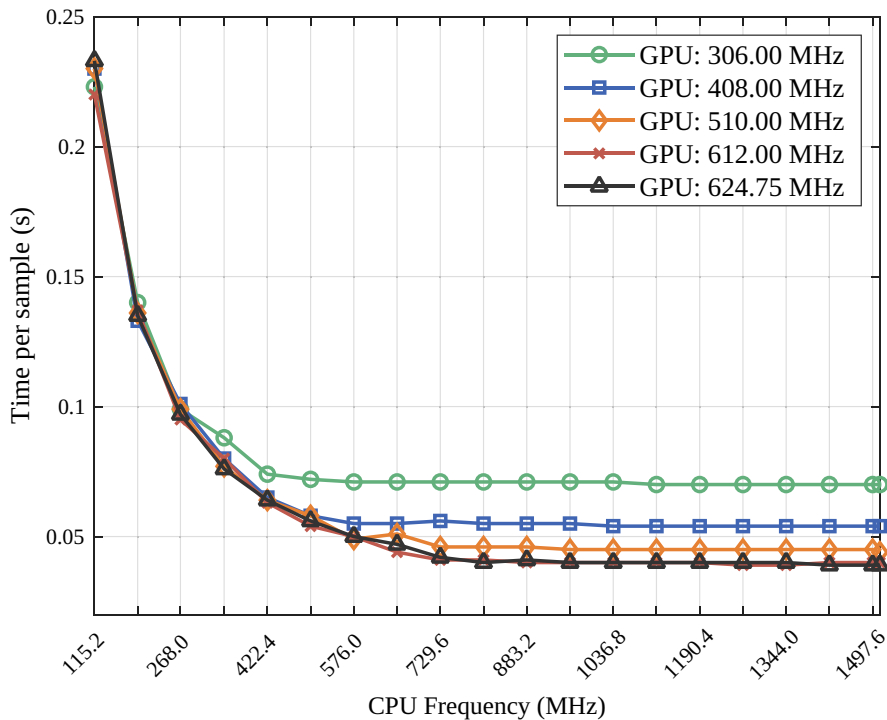


Fig. 6 Effect of varying CPU and GPU frequencies on the execution time per sample (in seconds) for GPU training with concurrent CPU inference

epoch, which consists of eight batches of 16 images each. The platform's average power consumption is calculated by averaging the power measurements taken every tenth of a second on the Jetson Orin module. As demonstrated in Fig. 5, power consumption is dependent on the computations performed by the module's various components, and its variability is linked to the CPU and GPU frequencies. For instance, Fig. 5a shows that for the minimum frequency profile, the average power consumption was 6.08 W with a standard deviation of 1.07 W. Conversely, in Fig. 5b (the maximum frequency profile), the average power consumption was 13.31 W, exhibiting a significantly lower standard deviation of 0.58 W. While Fig. 8 shows that the energy consumption per sample is relatively consistent across most CPU and GPU frequency settings, ranging from 0.28 to 0.41 joules. This consistency arises because the reduction in processing time from higher frequencies is largely offset by a proportional increase in power consumption. However, the trade-off is not perfect. The minimum energy consumption is achieved not at the frequency extremes, but at mid-range CPU frequencies (729.6–806.4 MHz) combined with the highest available GPU frequencies (612–624.75 MHz).

Therefore, these results reveal that simply maximizing CPU or GPU frequencies does not guarantee the most energy-efficient operation for on-device learning. Instead, a targeted mid-range frequency configuration proves to be a more effective option for energy-aware edge AI deployments.

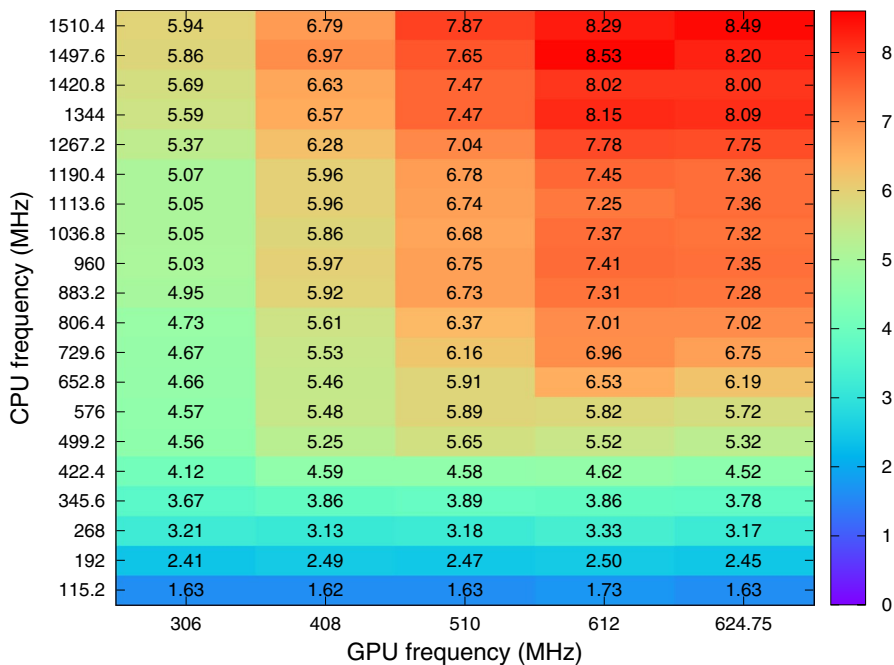


Fig. 7 Effect of varying CPU and GPU frequencies on the average power consumption (in watts) of the SOM during concurrent training and inference

6 Conclusion

In this paper, we presented a comprehensive study on enhancing the robustness of deep learning-based angle-of-arrival and angle-of-departure estimation in mmWave wireless systems under hardware impairments. We showed that fine-tuning a U-Net model with impairment-augmented data improves estimation accuracy, demonstrating the benefits of incorporating online learning into angle estimation tasks.

In addition, we characterized the energy-performance trade-offs of a co-execution strategy on the NVIDIA Jetson Orin Nano, where inference runs on the CPU while fine-tuning is carried out on the GPU. Our results highlight two main findings. First, impairment-aware fine-tuning effectively counters hardware degradation, particularly under significant phase impairments. In these scenarios, the fine-tuned model consistently preserves or even improves estimation accuracy, reducing angular error (RMSE) by approximately 3.6% and increasing the Probability of Detection (P_D) by up to 1 percentage point compared to the base model. Second, concurrent CPU inference and GPU training introduce a clear trade-off between time and energy: operating at maximum frequencies reduces training time by more than 11x (from 57.2s to 5.1s per two epochs) but also increases power consumption by over 5x. The most energy-efficient regime is achieved with mid-range CPU and high GPU frequencies, which keeps the energy per trained sample cost within 0.28 J. Specifically, the minimum energy consumption per

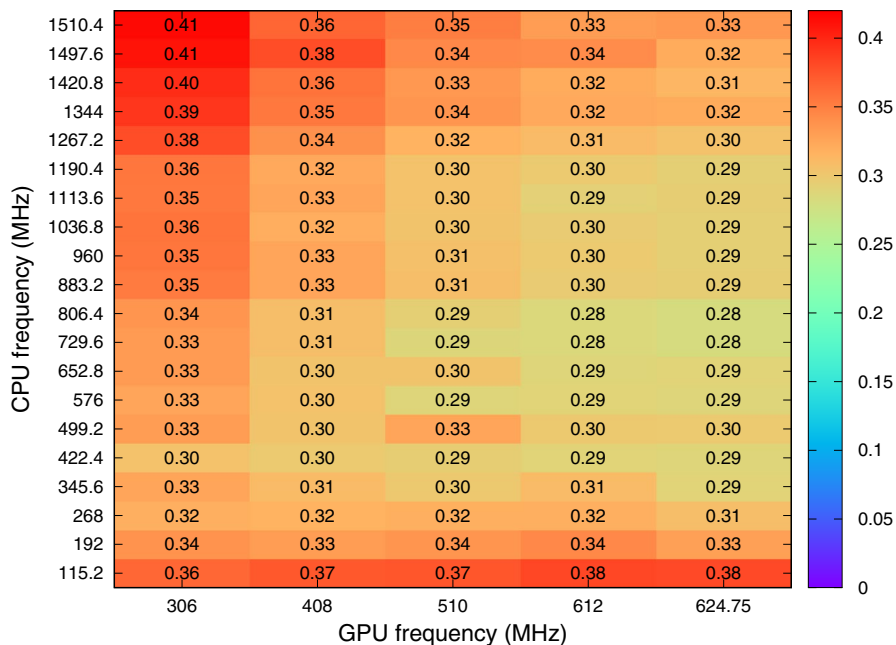


Fig. 8 Effect of varying CPU and GPU frequencies on the energy consumption per sample (in joules) of the SOM during concurrent training and inference

sample (0.282 J) is achieved by using the maximum GPU frequency (624.75 MHz) alongside a CPU frequency of either 729.6 MHz or 806.4 MHz.

Overall, our findings confirm the feasibility of concurrent training and inference on resource-constrained heterogeneous hardware. Future work will extend the proposed fine-tuning strategy to address other impairments, such as amplitude errors. Additionally, we are developing a ray-tracing simulator to validate the models in complex, deterministic environments with obstacles and reflections, moving beyond the stochastic data used here. Beyond the case of hardware impairments considered here, the proposed strategy represents a practical approach to enable continuous adaptation to environmental and hardware dynamics, paving the way for resilient and autonomous edge intelligence in future 6G systems.

Author contributions Conceptualization, E.M., J.M.B. and S.R.; methodology, E.M., J.M.B., and S.R.; software, E.M., S.V., J.M.B., and G.L.; validation, all authors; formal analysis, E.M., S.V., and J.M.B., E.M.; investigation, all authors; resources, all authors; data curation, E.M. and J.M.B.; original draft preparation, E.M. and J.M.B.; review and editing, all authors; visualization, E.M., S.V., and J.M.B.; supervision, S.R.; project administration, S.R.; funding acquisition, S.R. and C.B. All authors have read and agreed to the published version of the manuscript.

Funding Open Access funding provided thanks to the CRUE-CSIC agreement with Springer Nature. Thanks to Grants PID2023-148671OB-I00 and PLEC2024-011160 funded by MICIU/AEI

/10.13039/501100011033 and by ERDF/EU. Grants CIACIF/2023/416 and CIAICO/2022/179 from Generalitat Valenciana of Spain funded also partially this work.

Data availability No datasets were generated or analyzed during the current study.

Declarations

Conflict of interest The authors declare no Conflict of interest.

Ethical approval Not applicable.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Giordani M, Polese M, Mezzavilla M, Rangan S, Zorzi M (2020) Toward 6G networks: use cases and technologies. *IEEE Commun Mag* 58(3):55–61. <https://doi.org/10.1109/MCOM.001.1900411>
- Zeng Y, Chen J, Xu J, Wu D, Xu X, Jin S, Gao X, Gesbert D, Cui S, Zhang R (2024) A tutorial on environment-aware communications via channel knowledge map for 6G. *IEEE Commun Surv Tutor* 26(3):1478–1519. <https://doi.org/10.1109/COMST.2024.3364508>
- Andrews JG, Bai T, Kulkarni MN, Alkhateeb A, Gupta AK, Heath RW (2017) Modeling and analyzing millimeter wave cellular systems. *IEEE Trans Commun* 65(1):403–430. <https://doi.org/10.1109/TCOMM.2016.2618794>
- Giordani M, Polese M, Roy A, Castor D, Zorzi M (2019) A tutorial on beam management for 3GPP NR at mmWave frequencies. *IEEE Commun Surv Tutor* 21(1):173–196. <https://doi.org/10.1109/COMST.2018.2869411>
- Roger S, Cobos M, Botella-Mascarell C, Fodor G (2021) Fast channel estimation in the transformed spatial domain for analog millimeter wave systems. *IEEE Trans Wirel Commun* 20(9):5926–5941. <https://doi.org/10.1109/TWC.2021.3071315>
- Qin Z, Ye H, Li GY, Juang B-HF (2019) Deep learning in physical layer communications. *IEEE Wirel Commun* 26(2):93–99. <https://doi.org/10.1109/MWC.2019.1800601>
- Lloria D, Roger S, Botella-Mascarell C, Cobos M (2025) Deep-learning-based AoA and AoD estimation in analog millimeter wave MIMO systems. *IEEE Trans Veh Technol*. <https://doi.org/10.1109/TVT.2025.3637908>. (To appear in 2026)
- Lloria D, Roger S, León G, Badia JM, Botella-Mascarell C, Belloch JA (2024) Optimizing millimeter-wave MIMO channel estimation through GPU-based edge artificial intelligence. *J Supercomput* 81(1):270. <https://doi.org/10.1007/s11227-024-06795-2>
- Bakr O, Johnson M (2009) Impact of phase and amplitude errors on array performance. EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2009-1, 80
- Somjit N, Stemme G, Oberhammer J (2009) Phase error and nonlinearity investigation of millimeter-wave MEMS 7-stage dielectric-block phase shifters. In: 2009 European Microwave Conference, Vol 1–3, pp 1872–1875. <https://doi.org/10.1109/EUMC.2009.5296269>
- Moon T, Gaun J, Hassanieh H (2019) Online millimeter wave phased array calibration based on channel estimation. *IEEE*. <https://doi.org/10.1109/VTS.2019.8758627>
- Dimce S, Amjad MS, Dressler F (2021) mmWave on the road: investigating the weather impact on 60 GHz V2X communication channels. *IEEE*. <https://doi.org/10.23919/WONS51326.2021.9415572>

13. Mao Y, Yu X, Huang K, Angela Zhang Y-J, Zhang J (2024) Green edge AI: a contemporary survey. *Proc IEEE* 112(7):880–911. <https://doi.org/10.1109/JPROC.2024.3437365>
14. Zeng Q, Du Y, Huang K, Leung KK (2021) Energy-efficient resource management for federated edge learning with CPU-GPU heterogeneous computing. *IEEE Trans Wirel Commun* 20(12):7947–7962. <https://doi.org/10.1109/TWC.2021.3088910>
15. Alkhateeb A, El Ayach O, Leus G, Heath RW (2014) Channel estimation and hybrid precoding for millimeter wave cellular systems. *IEEE J Sel Top Signal Proc* 8(5):831–846. <https://doi.org/10.1109/JSTSP.2014.2334278>
16. Akdeniz MR, Liu Y, Samimi MK, Sun S, Rangan S, Rappaport TS, Erkip E (2014) Millimeter wave channel modeling and cellular capacity evaluation. *IEEE J Sel Areas Commun* 32(6):1164–1179. <https://doi.org/10.1109/JSAC.2014.2328154>
17. Zhang C, Guo D, Fan P (2016) Tracking angles of departure and arrival in a mobile millimeter wave channel. *IEEE*. <https://doi.org/10.1109/ICC.2016.7510902>
18. Park SC, Park MK, Kang MG (2003) Super-resolution image reconstruction: a technical overview. *IEEE Signal Process Mag* 20(3):21–36. <https://doi.org/10.1109/MSP.2003.1203207>
19. Pan P, Zhang Y, Deng Z, Qi W (2021) Deep learning-based 2-D frequency estimation of multiple sinusoids. *IEEE Trans Neural Netw Learn Syst* 33(10):5429–5440. <https://doi.org/10.1109/TNNLS.2021.3070707>
20. Lloria D, Roger S, Botella-Mascarell C, Cobos M, Villaescusa S (2025) DL_DOA: deep-learning-based AoA and AoD estimation in analog millimeter wave MIMO systems. GitHub. Accessed: 2025-12-09. <https://github.com/SandraRoger/DLDOA>
21. NVIDIA: NVIDIA Jetson Linux Developer Guide. Release 32.7.3. <https://docs.nvidia.com/jetson/archives/14t-archived/14t-3273/> (2022)
22. Barrachina S, Barreda M, Catalán S, Dolz MF, Fabregat G, Mayo R, Quintana-Ortí ES (2013) An integrated framework for power-performance analysis of parallel scientific workloads. In: *Proceedings of the Third International Conference on Smart Grids, Green Communications and IT Energy-Aware Technologies (ENERGY 2013)*

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Eric Meneses-Albalá¹ · Saúl Villaescusa¹ · José M. Badía² · Germán León² · Carmen Botella-Mascarell¹ · Sandra Roger¹

✉ Sandra Roger
sandra.roger@uv.es

Eric Meneses-Albalá
eric.meneses@uv.es

Saúl Villaescusa
saul.villaescusa@uv.es

José M. Badía
badia@uji.es

Germán León
leon@uji.es

Carmen Botella-Mascarell
carmen.botella@uv.es

¹ Computer Science Department, Universitat de València, Valencia, Spain

² HPCA, Universidad Jaume I de Castellón, Castellón de la Plana, Spain